5 Constructors and __init__



KEY IDEAS

- `Cls(args)` runs `__new__(cls, ...)` (allocates the object) and then `__init__(self, ...)` (initialises it). Override `__init__` almost always; `__new__` only for immutables and singletons.
- `__init__` must return `None`. It may take defaults, `*args`, `**kwargs` and validate input.
- Extra constructors: `@classmethod` factories. Add `__repr__` for debugging (`__str__` for end users).

PITFALL Never use a mutable default such as `items=[]`; it is built once and shared by every call. Use `items=None` and create the list inside: `self.items = [] if items is None else items`.

EXAMPLE

```
class Point:
    def __init__(self, x=0, y=0):
        self.x, self.y = x, y

    def __repr__(self):
        return f"Point({self.x}, {self.y})"

    @classmethod
    def from_str(cls, s):          # alternative constructor
        x, y = map(float, s.split(","))
        return cls(x, y)

print(Point())                     # → Point(0, 0)
print(Point(y=5))                  # → Point(0, 5)
print(Point.from_str("1,2"))       # → Point(1.0, 2.0)
```

```
class Single:                      # __new__ → singleton
    _inst = None
    def __new__(cls):
        if cls._inst is None:
            cls._inst = super().__new__(cls)
        return cls._inst

print(Single() is Single())        # → True
```